

Kimi:

I'll build a comprehensive **English** → **SQL Clause Map** first, then dissect each of the first 7 queries using that map.

English to SQL Clause Map

English Signal	SQL Clause/Keyword	When to Use
"Retrieve/Find/Display/Show/List/Get"	SELECT	Whenever you need to fetch/display data.
"names of students" / "all details"	SELECT column_name / SELECT *	* = all columns; specific names = only those columns.
"from table X"	FROM table_name	Specifies which table holds the data.
"enrolled in" / "belongs to" / "from society"	JOIN ... ON	When data spans multiple tables and you need to link them via a common column.
"whose" / "with" / "starting with" / "ending with" / "contains"	WHERE	Filters rows before grouping/aggregation.
"starts with 'A'" / "ends with '9'" / "has 'Gupta' in it"	WHERE ... LIKE 'A%' / LIKE '%9' / LIKE '%Gupta%'	Pattern matching: % = any characters, _ = single character.
"either X or Y"	WHERE (condition1 OR condition2)	When any one condition can be true.
"both X and Y"	WHERE condition1 AND condition2	When all conditions must be true simultaneously.
"course is A or B or C"	WHERE column IN ('A', 'B', 'C')	Cleaner alternative to multiple OR conditions for exact matches.
"more than N" / "less than N" / "at least"	WHERE column > N / < / >=	Comparison operators for numerical filtering.
"where N is input by user"	SET @variable = value;	User-defined variables in MySQL.
"update/modify/change"	UPDATE ... SET	Modifies existing data in a

English Signal	SQL Clause/Keyword	When to Use
		table.
“for a specific society” / “where id is X”	WHERE primary_key = 'X'	Targets exactly which row(s) to update.
“number of” / “count” / “how many”	COUNT(*) or COUNT(column)	Aggregate function — returns a single number.
“group by each society” / “for each course”	GROUP BY column	Groups rows so aggregates apply per group .
“having more than 5”	HAVING condition	Filters groups AFTER GROUP BY (unlike WHERE which filters rows before).
“youngest” / “oldest” / “highest” / “top”	ORDER BY column DESC/ASC + LIMIT n	Sorts results; DESC = descending (newest/largest first), LIMIT 1 = pick the top one.
“not enrolled in any”	LEFT JOIN ... WHERE ... IS NULL or NOT EXISTS	Finds missing relationships (anti-join pattern).
“all societies” / “every”	Compare COUNT(DISTINCT) against (SELECT COUNT(*) FROM table)	Division operation in relational algebra.

Queries 1–7: Detailed Breakdown

Q1. Retrieve names of students enrolled in any society.

English Breakdown:

English Fragment	SQL Mapping
“Retrieve names of students”	SELECT s.StudentName
“enrolled in any society”	Need data from both STUDENT and ENROLLMENT tables

Logical Thinking:

1. **What to show?** → Only student names (not all columns).
2. **Where is the data?** → Student names are in STUDENT; enrollment info is in ENROLLMENT.
3. **How to connect?** → A student “is enrolled” if their RollNo appears in ENROLLMENT. So link STUDENT.RollNo = ENROLLMENT.RollNo.
4. **“Any society”** → Just needs at least one match in ENROLLMENT. JOIN automatically keeps only matching rows (inner join behavior).
5. **Why DISTINCT?** → If a student enrolled in multiple societies, they’d appear multiple times. DISTINCT ensures each name appears once.

The Query:

```
SELECT DISTINCT s.StudentName
FROM STUDENT s
JOIN ENROLLMENT e ON s.RollNo = e.RollNo;
```

Execution Flow:

1. FROM STUDENT s → Start with STUDENT table (aliased as s).
 2. JOIN ENROLLMENT e ON s.RollNo = e.RollNo → For each student, find rows in ENROLLMENT with matching RollNo. Students with no enrollment are discarded.
 3. SELECT DISTINCT s.StudentName → Pick only the name column, removing duplicates.
-

Q2. Retrieve all society names.

English Breakdown:

English Fragment	SQL Mapping
“Retrieve”	SELECT
“all society names”	SocName from SOCIETY

Logical Thinking:

1. **What to show?** → Society names only.
2. **Where is the data?** → Only one table needed: SOCIETY.
3. **Any conditions?** → No filtering, no joining, no grouping. The simplest possible query.

The Query:

```
SELECT SocName FROM SOCIETY;
```

Q3. Retrieve students' names starting with the letter 'A'.

English Breakdown:

English Fragment	SQL Mapping
"Retrieve students' names"	SELECT StudentName FROM STUDENT
"starting with the letter 'A'"	WHERE StudentName LIKE 'A%'

Logical Thinking:

1. **Pattern matching signal:** "starting with" → LIKE 'A%'
 - A = first character must be A.
 - % = followed by any number of any characters (including zero).
2. **Why not =?** → = does exact match. 'A%' is a pattern, not a literal string.
3. **Case sensitivity:** In most MySQL configurations, LIKE is case-insensitive by default.

The Query:

```
SELECT StudentName FROM STUDENT WHERE StudentName LIKE 'A%';
```

Q4. Retrieve students' details studying in courses 'computer science' or 'chemistry'.

English Breakdown:

English Fragment	SQL Mapping
"Retrieve students' details"	SELECT * FROM STUDENT
"studying in courses 'computer science' or 'chemistry'"	WHERE Course = 'BSc CS' OR Course = 'BSc Chem'

Logical Thinking:

1. **"Details"** → * (all columns).
2. **"or"** → Two separate conditions combined with OR.
3. **Better alternative:** Since we're checking the **same column** against **multiple exact values**, use IN — it's cleaner and more scalable.

The Query:

– *Method 1: Using OR*

```
SELECT * FROM STUDENT WHERE Course = 'BSc CS' OR Course = 'BSc Chem';
```

-- Method 2: Using IN (preferred for readability)

SELECT * **FROM** STUDENT **WHERE** Course **IN** ('BSc CS', 'BSc Chem');

Why IN is better here:

- If you later add "or 'Physics'", you just add , 'BSc Phy' inside the parentheses instead of writing another OR clause.

Q5. Retrieve students' names whose roll no either starts with 'X' or 'Z' and ends with '9'.

English Breakdown:

English Fragment

SQL Mapping

"Retrieve students' names"

SELECT StudentName FROM STUDENT

"roll no either starts with 'X' or 'Z'"

(RollNo LIKE 'X%' OR RollNo LIKE 'Z%')

"and ends with '9'"

AND RollNo LIKE '%9'

Logical Thinking:

1. **"Either...or"** → Parentheses () are **critical** here. Without them, AND would bind tighter than OR, changing the meaning.
 - Wrong: RollNo LIKE 'X%' OR RollNo LIKE 'Z%' AND RollNo LIKE '%9' → This would match ALL X-starting roll numbers regardless of ending, and only Z-starting ones ending with 9.
 - Right: (RollNo LIKE 'X%' OR RollNo LIKE 'Z%') AND RollNo LIKE '%9' → Must end with 9 AND start with X or Z.
2. **Pattern anatomy:**
 - 'X%' = starts with X
 - '%9' = ends with 9
 - '%Gupta%' = contains Gupta (pattern for future reference)

The Query:

SELECT StudentName **FROM** STUDENT
WHERE (RollNo **LIKE** 'X%' **OR** RollNo **LIKE** 'Z%') **AND** RollNo **LIKE** '%9';

Q6. Find society details with more than N TotalSeats where N is to be input by the user.

English Breakdown:

English Fragment	SQL Mapping
"Find society details"	SELECT * FROM SOCIETY
"with more than N TotalSeats"	WHERE TotalSeats > N
"where N is to be input by the user"	SET @N = value;

Logical Thinking:

1. **"More than"** → > operator.
2. **User input in SQL:** SQL itself doesn't have interactive input in standard scripts. MySQL uses **user-defined variables** (@N).
 - First, set the variable: SET @N = 30;
 - Then use it in the query: WHERE TotalSeats > @N
3. **Why variable?** → If you hardcode 30, you'd need to rewrite the query for 40. With @N, you change one line.

The Query:

```
SET @N = 30;  
SELECT * FROM SOCIETY WHERE TotalSeats > @N;
```

Execution Flow:

1. MySQL stores 30 in session variable @N.
2. Query reads @N and substitutes it during execution.
3. Only rows where TotalSeats is greater than 30 are returned.

Q7. Update society table for the mentor name of a specific society.

English Breakdown:

English Fragment	SQL Mapping
"Update society table"	UPDATE SOCIETY
"for the mentor name"	SET MentorName = 'new value'
"of a specific society"	WHERE SocID = 'S1'

Logical Thinking:

1. **“Update”** → This is a **DML (Data Manipulation Language)** command, not a SELECT. It **modifies** existing data.
2. **Structure:** UPDATE table SET column = new_value WHERE condition
3. **The WHERE is mandatory here** → Without it, you'd change the mentor name for **ALL societies** in the table! The WHERE pinpoints exactly which row(s) to affect.
4. **“Specific society”** → Identified by primary key SocID = 'S1'.

The Query:

UPDATE SOCIETY **SET** MentorName = 'Prof. R Sharma' **WHERE** SocID = 'S1';

What happens:

1. MySQL finds the row in SOCIETY where SocID is 'S1'.
2. It changes only the MentorName column in that row to 'Prof. R Sharma'.
3. All other columns and rows remain untouched.

Summary Cheat Sheet (First 7 Queries)

Query	Key Clauses Used	Key Concept
Q1	SELECT, JOIN, DISTINCT	Multi-table fetch, duplicate removal
Q2	SELECT, FROM	Single-table projection
Q3	SELECT, WHERE, LIKE	Pattern matching with %
Q4	SELECT, WHERE, OR/IN	Multi-value filtering
Q5	SELECT, WHERE, LIKE, AND, OR	Compound conditions + parentheses
Q6	SET variable, SELECT, WHERE, >	User-defined variables + comparison
Q7	UPDATE, SET, WHERE	Data modification with precise targeting

Ready for the next block of 7 queries (Q8–Q14)! Just say the word and I'll break them down with the same level of detail — including GROUP BY, HAVING, subqueries, and aggregation logic.

Kimi:

Here are the next 7 queries (Q8–Q14) broken down with the **English** → **SQL Clause Map** and detailed reasoning.

Extended English to SQL Clause Map (New Concepts for Q8–Q14)

English Signal	SQL Clause/Keyword	When to Use
“more than five students have enrolled” / “at least two”	GROUP BY + HAVING COUNT(...) >= N	Aggregation + group filtering: First group rows, then filter groups based on aggregate count.
“for each society” / “per society”	GROUP BY SocID, SocName	Required whenever you calculate something per group rather than across the whole table.
“most popular” / “youngest” / “oldest” / “top”	ORDER BY ... DESC + LIMIT n	Sort results in descending order and pick the first <i>n</i> .
“least popular” / “oldest first”	ORDER BY ... ASC + LIMIT n	Sort in ascending order and pick the first <i>n</i> .
“not enrolled in any” / “no students”	LEFT JOIN ... WHERE ... IS NULL	Anti-join pattern: Keep all rows from one table even if no match exists, then filter where the match is missing.
“maximum students are enrolled”	Subquery: (SELECT MAX(CountCol) FROM ...)	When you need to compare against a computed maximum value from another query.
“youngest student”	ORDER BY DOB DESC LIMIT 1	Youngest = most recent date of birth = highest date value.

Q8. Find society names in which more than five students have enrolled.

English Breakdown:

English Fragment

SQL Mapping

“Find society names”

SELECT s.SocName

“in which more than five students have enrolled”

GROUP BY s.SocID, s.SocName + HAVING
COUNT(e.RollNo) > 5

Logical Thinking:

1. **“More than five students have enrolled”** → This is an **aggregate condition**. You cannot use WHERE because WHERE filters individual rows *before* grouping. You need to count enrollments **per society** first, then check if that count exceeds 5.
2. **Step 1: Join tables** → Need SOCIETY (for names) and ENROLLMENT (for enrollment records).
3. **Step 2: Group by society** → GROUP BY s.SocID, s.SocName collapses all enrollment rows for each society into a single summary row.
4. **Step 3: Filter groups** → HAVING COUNT(e.RollNo) > 5 keeps only those groups where the enrollment count exceeds 5.

Critical Rule: WHERE filters **rows** before grouping. HAVING filters **groups** after aggregation.

Whenever you see “more than X [things] per [category]”, use GROUP BY + HAVING.

The Query:

```
SELECT s.SocName
FROM SOCIETY s
JOIN ENROLLMENT e ON s.SocID = e.SID
GROUP BY s.SocID, s.SocName
HAVING COUNT(e.RollNo) > 5;
```

Execution Flow:

1. JOIN matches societies with their enrollments.
 2. GROUP BY bundles rows by society (e.g., all NSS enrollments become one group).
 3. COUNT(e.RollNo) calculates how many students per group.
 4. HAVING discards groups with count ≤ 5.
 5. SELECT returns only the surviving society names.
-

Q9. Find the name of the youngest student enrolled in society 'NSS'.

English Breakdown:

English Fragment

SQL Mapping

"Find the name of the youngest student"

SELECT s.StudentName + ORDER BY s.DOB DESC
LIMIT 1

"enrolled in society 'NSS'"

JOIN all three tables + WHERE so.SocName =
'NSS'

Logical Thinking:

1. **"Youngest student"** → In terms of DOB, the youngest person has the **most recent** date of birth (e.g., 2002 > 1999). So sort by DOB DESC (descending = newest first) and pick the top 1.
2. **Three-table join needed:**
 - STUDENT → to get the name and DOB.
 - ENROLLMENT → to know who is enrolled.
 - SOCIETY → to filter only 'NSS'.
3. **"The youngest" = single result** → LIMIT 1 ensures only one row is returned.

Age vs. DOB Trap: If the question said "oldest student", you'd use ORDER BY DOB ASC LIMIT 1 (earliest date = oldest person). "Youngest" always means the latest DOB.

The Query:

```
SELECT s.StudentName
FROM STUDENT s
JOIN ENROLLMENT e ON s.RollNo = e.RollNo
JOIN SOCIETY so ON e.SID = so.SocID
WHERE so.SocName = 'NSS'
ORDER BY s.DOB DESC
LIMIT 1;
```

Execution Flow:

1. Join all three tables to connect students → enrollments → society names.
 2. WHERE so.SocName = 'NSS' keeps only NSS enrollments.
 3. ORDER BY s.DOB DESC arranges NSS students from youngest to oldest.
 4. LIMIT 1 extracts just the first (youngest) student.
-

Q10. Find the name of the most popular society (on the basis of enrolled students).

English Breakdown:

English Fragment

SQL Mapping

“Find the name of the most popular society”

SELECT s.SocName

“on the basis of enrolled students”

COUNT(e.RollNo) as popularity metric

“most popular” = highest count

ORDER BY COUNT(...) DESC LIMIT 1

Logical Thinking:

1. **“Most popular”** → The society with the **maximum number of enrollments**.
2. **Count enrollments per society** → GROUP BY + COUNT(e.RollNo).
3. **Pick the top** → Sort by that count in descending order, take LIMIT 1.
4. **Why LEFT JOIN?** → The sample query uses LEFT JOIN to include societies with zero enrollments in the ranking (though LIMIT 1 with DESC will still pick the highest). LEFT JOIN ensures no society is silently excluded.

The Query:

```
SELECT s.SocName, COUNT(e.RollNo) AS StudentCount
FROM SOCIETY s
LEFT JOIN ENROLLMENT e ON s.SocID = e.SID
GROUP BY s.SocID, s.SocName
ORDER BY StudentCount DESC
LIMIT 1;
```

Execution Flow:

1. LEFT JOIN pairs every society with its enrollments (societies with no students still appear with NULLs).
 2. GROUP BY aggregates per society.
 3. COUNT(e.RollNo) computes total students per society (NULLs don't count).
 4. ORDER BY StudentCount DESC places the society with the highest enrollment at the top.
 5. LIMIT 1 returns only that top society.
-

Q11. Find the name of two least popular societies (on the basis of enrolled students).

English Breakdown:

English Fragment

SQL Mapping

“Find the name of two least popular societies”

SELECT s.SocName + LIMIT 2

“least popular”

ORDER BY COUNT(e.RollNo) ASC

Logical Thinking:

1. **“Least popular”** → Lowest enrollment count. Opposite of Q10.
2. **“Two”** → LIMIT 2 instead of LIMIT 1.
3. **Sort ascending** → ASC puts the smallest counts first.

Comparison with Q10: This is structurally identical to Q10 except DESC becomes ASC, and LIMIT 1 becomes LIMIT 2.

The Query:

```
SELECT s.SocName, COUNT(e.RollNo) AS StudentCount
FROM SOCIETY s
LEFT JOIN ENROLLMENT e ON s.SocID = e.SID
GROUP BY s.SocID, s.SocName
ORDER BY StudentCount ASC
LIMIT 2;
```

Q12. Find the students names who are not enrolled in any society.

English Breakdown:

English Fragment

SQL Mapping

“Find the students names”

SELECT s.StudentName

“who are not enrolled in any society”

LEFT JOIN ... WHERE e.RollNo IS NULL

Logical Thinking:

1. **“Not enrolled in any”** → This is a **negative condition** on a relationship. You need students who exist in STUDENT but have **no matching row** in ENROLLMENT.
2. **The Anti-Join Pattern:**
 - Use LEFT JOIN (not INNER JOIN) to keep **all students**, even those without enrollments.
 - For non-enrolled students, all ENROLLMENT columns will be NULL.

- Filter with WHERE e.RollNo IS NULL to keep only those unmatched rows.

3. **Alternative:** NOT EXISTS achieves the same result and is sometimes preferred for performance on large datasets.

The Query:

– *Method 1: LEFT JOIN + IS NULL (most common)*

```
SELECT s.StudentName
FROM STUDENT s
LEFT JOIN ENROLLMENT e ON s.RollNo = e.RollNo
WHERE e.RollNo IS NULL;
```

– *Method 2: NOT EXISTS (semantically clearer for "none")*

```
SELECT StudentName FROM STUDENT s
WHERE NOT EXISTS (
    SELECT 1 FROM ENROLLMENT e WHERE e.RollNo = s.RollNo
);
```

Why LEFT JOIN works here:

- INNER JOIN would discard students with no enrollments immediately.
- LEFT JOIN preserves them with NULL in the right table columns.
- WHERE e.RollNo IS NULL catches exactly those preserved-but-unmatched rows.

Q13. Find the students names enrolled in at least two societies.

English Breakdown:

English Fragment	SQL Mapping
"Find the students names"	SELECT s.StudentName
"enrolled in at least two societies"	GROUP BY s.RollNo, s.StudentName + HAVING COUNT(DISTINCT e.SID) >= 2

Logical Thinking:

1. **"At least two societies"** → Aggregate condition per student. Requires GROUP BY on student + HAVING.
2. **Why COUNT(DISTINCT e.SID)?** → A student might have accidentally enrolled twice in the same society (duplicate rows). DISTINCT ensures each society is counted only once, making the logic robust.
3. **"At least"** → >= 2. "Exactly two" would be = 2. "More than two" would be > 2.

The Query:

```
SELECT s.StudentName
FROM STUDENT s
JOIN ENROLLMENT e ON s.RollNo = e.RollNo
GROUP BY s.RollNo, s.StudentName
HAVING COUNT(DISTINCT e.SID) >= 2;
```

Execution Flow:

1. Join students with their enrollments.
 2. Group all enrollment rows by each unique student.
 3. Count how many **distinct** societies appear in each group.
 4. HAVING >= 2 keeps only students meeting the threshold.
 5. Return their names.
-

Q14. Find society names in which maximum students are enrolled.

English Breakdown:

English Fragment	SQL Mapping
"Find society names"	SELECT s.SocName
"in which maximum students are enrolled"	HAVING EnrolledCount = (SELECT MAX(...) FROM ...)

Logical Thinking:

1. **"Maximum students are enrolled"** → This is a **superlative** condition. You cannot simply sort and limit because you need to handle ties (multiple societies might have the same maximum count).
2. **Two-step logic:**
 - **Step A:** Calculate the enrollment count for every society.
 - **Step B:** Find the **maximum value** among all those counts.
 - **Step C:** Return all societies whose count equals that maximum.
3. **Subquery structure:**
 - Outer query: Groups societies and counts enrollments.
 - Inner query: Finds the maximum enrollment count across all groups.
 - HAVING compares each group's count against that inner query result.

The Query:

```
SELECT s.SocName, COUNT(e.RollNo) AS EnrolledCount
FROM SOCIETY s
JOIN ENROLLMENT e ON s.SocID = e.SID
GROUP BY s.SocID, s.SocName
HAVING EnrolledCount = (
    SELECT MAX(StudentCount)
    FROM (
        SELECT COUNT(RollNo) AS StudentCount
        FROM ENROLLMENT
        GROUP BY SID
    ) AS counts
);
```

Subquery Breakdown:

Subquery Level	Purpose
Innermost SELECT COUNT(RollNo) ... GROUP BY SID	Creates a temporary table of enrollment counts per society (e.g., NSS: 4, Debating: 3, etc.).
Middle SELECT MAX(StudentCount) FROM (...)	Finds the single highest number from that temporary table (e.g., 4).
Outer HAVING EnrolledCount = (...)	Compares each society’s count against that maximum. If equal, the society is returned.

Why not just ORDER BY ... DESC LIMIT 1? → LIMIT 1 would return only one society even if two societies are tied for maximum. The subquery approach with HAVING returns **all ties**, which is more accurate for “maximum”.

Summary Cheat Sheet (Q8–Q14)

Query	Key Clauses Used	Key Concept
Q8	JOIN, GROUP BY, HAVING COUNT > 5	Aggregate filtering per group
Q9	JOIN (3 tables), WHERE, ORDER BY DESC, LIMIT 1	Superlative with date sorting
Q10	LEFT JOIN, GROUP BY, ORDER BY DESC, LIMIT 1	Maximum popularity
Q11	LEFT JOIN, GROUP BY, ORDER BY ASC, LIMIT	Minimum popularity (bottom N)

Query	Key Clauses Used	Key Concept
	2	
Q12	LEFT JOIN, WHERE IS NULL / NOT EXISTS	Anti-join (missing relationships)
Q13	JOIN, GROUP BY, HAVING COUNT DISTINCT >= 2	Threshold counting per entity
Q14	JOIN, GROUP BY, HAVING = (SELECT MAX(...) FROM ...)	Subquery for superlative with tie handling

Ready for Q15–Q21! Just prompt me and I’ll continue with the next block.

Kimi:

Here are queries **Q15–Q21** with the extended **English** → **SQL Clause Map** and detailed breakdowns.

Extended English to SQL Clause Map (New Concepts for Q15–Q21)

English Signal	SQL Clause/Keyword	When to Use
“names of students and society names”	SELECT col1, col2 from multiple joined tables	When the answer requires columns from different tables in the same result row.
“any of the three societies X, Y, Z”	WHERE column IN ('X', 'Y', 'Z')	Cleaner than multiple OR conditions when checking one column against several exact values.
“has ‘Gupta’ in it” / “contains”	WHERE column LIKE '%Gupta%'	Pattern matching for substring anywhere in the text.
“only 10% of its capacity”	HAVING aggregate = column * 0.10	Aggregate comparison involving arithmetic with another column .
“vacant seats” / “unfilled” / “remaining”	column1 - IFNULL(column2, 0)	Computed column; IFNULL/COALESCE handles cases where there is no match (NULL becomes 0).

English Signal	SQL Clause/Keyword	When to Use
“for each society”	GROUP BY (or join with derived table)	Per-group calculation.
“Increment ... by 10%”	UPDATE table SET col = col * 1.1	Arithmetic update affecting all rows (no WHERE = every row).
“Add ... field / column”	ALTER TABLE ... ADD COLUMN ...	DDL (Data Definition Language) — changes table structure, not data.

Q15. Find names of all students who have enrolled in any society and society names in which at least one student has enrolled.

English Breakdown:

English Fragment	SQL Mapping
“Find names of all students”	SELECT s.StudentName
“and society names”	, so.SocName
“who have enrolled in any society”	JOIN ENROLLMENT (student side)
“in which at least one student has enrolled”	JOIN ENROLLMENT (society side)

Logical Thinking:

1. **Two things requested in one result:** student name + society name. This means the output is a **list of pairs**.
2. **Where do pairs exist?** → Only in ENROLLMENT. Each row in ENROLLMENT says “Student X is in Society Y”.
3. **Need all three tables:**
 - STUDENT → to convert RollNo into StudentName.
 - SOCIETY → to convert SocID into SocName.
 - ENROLLMENT → the bridge that connects them.
4. **Why DISTINCT?** → Though the PK on ENROLLMENT prevents exact duplicates, DISTINCT is a safety net ensuring each (Student, Society) pair appears once.

Mental Model: Think of ENROLLMENT as the middle of a bridge. You need to walk from STUDENT across ENROLLMENT to SOCIETY to answer this.

The Query:

```
SELECT DISTINCT s.StudentName, so.SocName
FROM STUDENT s
JOIN ENROLLMENT e ON s.RollNo = e.RollNo
JOIN SOCIETY so ON e.SID = so.SocID;
```

Execution Flow:

1. JOIN STUDENT ENROLLMENT → links students to their enrollment records.
 2. JOIN ENROLLMENT SOCIETY → links those records to society names.
 3. SELECT DISTINCT s.StudentName, so.SocName → outputs clean pairs.
-

Q16. Find names of students who are enrolled in any of the three societies 'Debating', 'Dancing' and 'Sashakt'.

English Breakdown:

English Fragment	SQL Mapping
"Find names of students"	SELECT DISTINCT s.StudentName
"enrolled in any of the three societies"	WHERE so.SocName IN ('Debating', 'Dancing', 'Sashakt')

Logical Thinking:

1. **"Any of the three"** → This is a classic IN scenario. You are checking the same column (SocName) against a list of specific values.
2. **Alternative:** WHERE so.SocName = 'Debating' OR so.SocName = 'Dancing' OR so.SocName = 'Sashakt' works but is verbose and error-prone.
3. **Need 3-table join** again because the filter is on SOCIETY.SocName but the output is STUDENT.StudentName.
4. **DISTINCT** is important here — a student enrolled in both Debating and Dancing would otherwise appear twice.

Rule of Thumb: When you see "any of...", "one of...", "either X, Y, or Z" applied to the **same column**, use IN (list).

The Query:

```
SELECT DISTINCT s.StudentName
FROM STUDENT s
```

JOIN ENROLLMENT e ON s.RollNo = e.RollNo
 JOIN SOCIETY so ON e.SID = so.SocID
 WHERE so.SocName IN ('Debating', 'Dancing', 'Sashakt');

Q17. Find society names such that its mentor has a name with 'Gupta' in it.

English Breakdown:

English Fragment	SQL Mapping
"Find society names"	SELECT SocName FROM SOCIETY
"mentor has a name with 'Gupta' in it"	WHERE MentorName LIKE '%Gupta%'

Logical Thinking:

1. **"Has 'Gupta' in it"** → Substring match. The word 'Gupta' can appear at the start, middle, or end of the name.
 - 'Gupta%' = starts with Gupta (wrong — misses "Dr. R Gupta").
 - '%Gupta' = ends with Gupta (wrong — misses "Gupta Ji").
 - '%Gupta%' = contains Gupta anywhere (correct).
2. **Single table only** → Both SocName and MentorName are in SOCIETY. No join needed.

The Query:

SELECT SocName FROM SOCIETY WHERE MentorName LIKE '%Gupta%';

Q18. Find the society names in which the number of enrolled students is only 10% of its capacity.

English Breakdown:

English Fragment	SQL Mapping
"Find the society names"	SELECT s.SocName
"number of enrolled students"	COUNT(e.RollNo)
"is only 10% of its capacity"	HAVING COUNT(e.RollNo) = s.TotalSeats * 0.1

Logical Thinking:

1. **"10% of its capacity"** → Mathematical expression: $\text{TotalSeats} \times 0.10$. This is a **per-society** calculation, so it happens after GROUP BY.

2. **Aggregate vs. Column comparison** → You are comparing COUNT(...) (aggregate) with s.TotalSeats * 0.1 (column arithmetic). This is perfectly valid inside HAVING.
3. **“Only”** → Exact equality (=), not >= or <=.

Important: WHERE cannot compare aggregates with columns. HAVING is the only place where COUNT(...) can be compared to TotalSeats * 0.1.

The Query:

```
SELECT s.SocName
FROM SOCIETY s
JOIN ENROLLMENT e ON s.SocID = e.SID
GROUP BY s.SocID, s.SocName, s.TotalSeats
HAVING COUNT(e.RollNo) = s.TotalSeats * 0.1;
```

Execution Flow:

1. Join societies with enrollments.
 2. Group by each unique society (including TotalSeats in GROUP BY because it appears in SELECT/HAVING).
 3. Count enrollments per group.
 4. HAVING checks if that count exactly equals 10% of capacity.
 5. Return matching society names.
-

Q19. Display the vacant seats for each society.

English Breakdown:

English Fragment	SQL Mapping
“Display the vacant seats”	SELECT ..., TotalSeats - EnrolledCount AS VacantSeats
“for each society”	Per-society calculation
“vacant” = capacity minus occupied	Arithmetic with IFNULL to treat missing enrollments as 0

Logical Thinking:

1. **Vacant seats formula:** TotalSeats – CurrentlyEnrolled.
2. **Problem:** Some societies might have **zero** enrollments. If you simply do TotalSeats - COUNT(...), a society with no students won’t appear in an inner join, or the count will be NULL.

3. Solution — LEFT JOIN with derived table:

- First, pre-calculate enrollment counts per society in a subquery (derived table).
- Then LEFT JOIN that result to SOCIETY so every society appears.
- Use IFNULL(EnrolledCount, 0) to convert NULL (no enrollments) into 0.

4. **Why derived table?** → You need to aggregate enrollments *before* joining, otherwise you'd have to use GROUP BY on the outer query. Both approaches work; the derived table approach is often cleaner for subtraction logic.

The Query:

```
SELECT SocName, TotalSeats - IFNULL(EnrolledCount, 0) AS VacantSeats
FROM SOCIETY s
LEFT JOIN (
  SELECT SID, COUNT(RollNo) AS EnrolledCount
  FROM ENROLLMENT
  GROUP BY SID
) e ON s.SocID = e.SID;
```

Subquery Breakdown:

Part	Purpose
(SELECT SID, COUNT(RollNo) ... GROUP BY SID)	Creates a mini-table of society IDs and their enrollment counts.
LEFT JOIN ... ON s.SocID = e.SID	Matches each society to its count. Societies with no enrollments still appear, but EnrolledCount is NULL.
IFNULL(EnrolledCount, 0)	Converts NULL to 0 so subtraction works.
TotalSeats - ...	Computes vacant seats.

Alternative mental model: Think of the derived table as a “pre-computed lookup” that you attach to the main table.

Q20. Increment Total Seats of each society by 10%.

English Breakdown:

English Fragment	SQL Mapping
“Increment”	UPDATE
“Total Seats of each society”	SET TotalSeats = ...

English Fragment	SQL Mapping
"by 10%"	TotalSeats * 1.1

Logical Thinking:

1. **"Increment by 10%"** → New value = Old value + 10% of old value = Old × 1.1.
2. **"Each society"** → No WHERE clause. Without WHERE, UPDATE affects **every row** in the table.
3. **Data type caution:** If TotalSeats is INT UNSIGNED, multiplying by 1.1 may produce decimals. MySQL will truncate or round depending on context. For exact integer results, you might use TotalSeats + (TotalSeats * 0.1) or ensure proper casting.

The Query:

UPDATE SOCIETY **SET** TotalSeats = TotalSeats * 1.1;

What happens:

1. MySQL scans every row in SOCIETY.
2. For each row, it reads TotalSeats, multiplies by 1.1, and writes it back.
3. All societies are updated simultaneously.

Q21. Add the enrollment fees paid ('yes'/'No') field in the enrollment table.

English Breakdown:

English Fragment	SQL Mapping
"Add ... field"	ALTER TABLE ... ADD COLUMN
"in the enrollment table"	ALTER TABLE ENROLLMENT
"with values 'yes'/'No'"	VARCHAR(3) or ENUM('Yes','No')
"default 'No'"	DEFAULT 'No'

Logical Thinking:

1. **"Add field"** → This changes the **structure/schema** of the table, not the data inside rows. This is **DDL (Data Definition Language)**, not DML.
2. **Command structure:** ALTER TABLE table_name ADD COLUMN column_name data_type constraints;
3. **Data type choice:** Since values are 'Yes'/'No', VARCHAR(3) is used. ENUM('Yes','No') would be even stricter but VARCHAR is more universally taught.

4. **Default value:** Existing rows in the table need a value for this new column. DEFAULT 'No' automatically fills it in.

The Query:

```
ALTER TABLE ENROLLMENT ADD COLUMN FeesPaid VARCHAR(3) DEFAULT 'No';
```

What happens:

1. MySQL modifies the table schema to include a new column FeesPaid.
2. All **existing rows** automatically get the value 'No'.
3. Future INSERT statements can specify 'Yes' or 'No'.

Summary Cheat Sheet (Q15–Q21)

Query	Key Clauses Used	Key Concept
Q15	JOIN (3 tables), SELECT col1, col2, DISTINCT	Multi-column fetch across bridge table
Q16	JOIN (3 tables), WHERE ... IN (...)	Filtering against a list of values
Q17	SELECT, WHERE ... LIKE '%...%'	Substring pattern matching
Q18	JOIN, GROUP BY, HAVING aggregate = column * 0.1	Aggregate-to-column arithmetic comparison
Q19	LEFT JOIN with derived table , IFNULL, arithmetic	Pre-aggregation + vacancy calculation
Q20	UPDATE ... SET col = col * 1.1 (no WHERE)	Bulk arithmetic update
Q21	ALTER TABLE ... ADD COLUMN ... DEFAULT	Schema modification (DDL)

Ready for Q22–Q28! Just send the next prompt and I'll continue.

Kimi:

Here are queries **Q22–Q28** with the extended **English** → **SQL Clause Map** and detailed breakdowns.

Extended English to SQL Clause Map (New Concepts for Q22–Q28)

English Signal	SQL Clause/Keyword	When to Use
"Update X to A, Y to B, Z to C"	Multiple UPDATE ... WHERE	When different rows need

English Signal	SQL Clause/Keyword	When to Use
	statements OR CASE WHEN	different values based on their identity.
“current date” / “today”	CURDATE() or CURRENT_DATE	Built-in function returning today’s date.
“Create a view”	CREATE VIEW name AS SELECT ...	To save a reusable query as a virtual table.
“enrolled in all the societies”	GROUP BY + HAVING COUNT(DISTINCT ...) = (SELECT COUNT(*) FROM ...)	Relational division : compare a student’s distinct count against the total available.
“Count the number of societies with...”	SELECT COUNT(*) FROM (subquery)	Wrap the filtering query inside a counter .
“Add column ... with default value”	ALTER TABLE ... ADD COLUMN ... DEFAULT ...	Schema change to introduce a new attribute with a fallback value.
“age is > 20 years”	TIMESTAMPDIFF(YEAR, DOB, CURDATE()) > 20	Proper age calculation (not just year subtraction).
“born in 2001”	YEAR(DOB) = 2001	Extract year from a DATE column.
“and are enrolled in at least one”	JOIN or EXISTS	Combine a time/attribute filter with a relationship filter.

Q22. Update date of enrollment of society id ‘s1’ to ‘2018-01-15’, ‘s2’ to the current date and ‘s3’ to ‘2018-01-02’.

English Breakdown:

English Fragment	SQL Mapping
“Update date of enrollment”	UPDATE ENROLLMENT SET DateOfEnrollment = ...
“of society id ‘s1’ to ‘2018-01-15’”	WHERE SID = 'S1' → specific date string
“‘s2’ to the current date”	WHERE SID = 'S2' → CURDATE()

“s3” to ‘2018-01-02’”

WHERE SID = 'S3' → specific date string

Logical Thinking:

1. **Different rows, different values** → Since each society ID needs a **different date**, you need **separate UPDATE statements** (one per condition) OR a single UPDATE with a CASE expression.
2. **“Current date”** → CURDATE() is MySQL’s built-in function that returns YYYY-MM-DD of the server’s current date. It makes the query dynamic — running it tomorrow gives tomorrow’s date.
3. **Why separate statements in the sample?** → For clarity and beginner readability. Each WHERE clause isolates exactly one set of rows.
4. **Better production approach** (for understanding):

```
UPDATE ENROLLMENT
SET DateOfEnrollment = CASE SID
    WHEN 'S1' THEN '2018-01-15'
    WHEN 'S2' THEN CURDATE()
    WHEN 'S3' THEN '2018-01-02'
END
WHERE SID IN ('S1', 'S2', 'S3');
```

This hits the table **once** instead of three times.

The Query (as given):

```
UPDATE ENROLLMENT SET DateOfEnrollment = '2018-01-15' WHERE SID = 'S1';
UPDATE ENROLLMENT SET DateOfEnrollment = CURDATE() WHERE SID = 'S2';
UPDATE ENROLLMENT SET DateOfEnrollment = '2018-01-02' WHERE SID = 'S3';
```

Execution Flow:

1. First statement: Finds all rows in ENROLLMENT where SID = 'S1' and sets their date to 2018-01-15.
 2. Second statement: Finds all rows where SID = 'S2' and sets their date to today’s date.
 3. Third statement: Finds all rows where SID = 'S3' and sets their date to 2018-01-02.
-

Q23. Create a view to keep track of society names with the total number of students enrolled in it.

English Breakdown:

English Fragment

SQL Mapping

“Create a view”

CREATE VIEW view_name AS

“to keep track of”

The view encapsulates a query you will run repeatedly.

“society names with the total number of students enrolled”

SELECT SocName, COUNT(RollNo) grouped by society.

Logical Thinking:

1. **What is a VIEW?** → A **virtual table** that stores a SELECT query, not actual data. Every time you query the view, the underlying SQL runs fresh.
2. **Why use a view here?** → “Keep track of” implies this is a report you’ll want to check frequently. Instead of rewriting the JOIN + GROUP BY every time, you save it as Society_Enrollment_Count.
3. **Structure:** CREATE VIEW name AS (your SELECT query);
4. **Querying the view:** After creation, you treat it exactly like a table: SELECT * FROM Society_Enrollment_Count;

The Query:

```
CREATE VIEW Society_Enrollment_Count AS
SELECT s.SocName, COUNT(e.RollNo) AS TotalEnrolled
FROM SOCIETY s
LEFT JOIN ENROLLMENT e ON s.SocID = e.SID
GROUP BY s.SocID, s.SocName;
```

– To use the view:

```
SELECT * FROM Society_Enrollment_Count;
```

Execution Flow:

1. CREATE VIEW stores the definition in the database schema.
2. When you later SELECT * FROM Society_Enrollment_Count, MySQL internally executes the saved JOIN + GROUP BY.
3. The result is computed in real-time, so it always reflects the latest data.

Q24. Find student names enrolled in all the societies.

English Breakdown:

English Fragment

SQL Mapping

“Find student names”

SELECT s.StudentName

“enrolled in **all** the societies”

GROUP BY s.RollNo + HAVING COUNT(DISTINCT
e.SID) = (SELECT COUNT(*) FROM SOCIETY)

Logical Thinking:

1. **“All the societies”** → This is the classic **Relational Division** problem in SQL.
 - You cannot simply write WHERE SocName = 'all' because there is no such value.
 - Instead, you count how many **distinct** societies each student is in, and check if that equals the **total number of societies available**.
2. **Logic equation:**
 - If there are 6 societies total, and Student X is enrolled in 6 distinct societies, then Student X is enrolled in **all** of them.
3. **Why COUNT(DISTINCT e.SID)?** → Prevents duplicate enrollments in the same society from inflating the count.
4. **The subquery** (SELECT COUNT(*) FROM SOCIETY) gives the denominator (total societies). The student's count is the numerator.

The Query:

```
SELECT s.StudentName
FROM STUDENT s
JOIN ENROLLMENT e ON s.RollNo = e.RollNo
GROUP BY s.RollNo, s.StudentName
HAVING COUNT(DISTINCT e.SID) = (SELECT COUNT(*) FROM SOCIETY);
```

Subquery Breakdown:

Part	Purpose
(SELECT COUNT(*) FROM SOCIETY)	Returns the total number of societies in the database (e.g., 6).
COUNT(DISTINCT e.SID)	Returns how many unique societies this particular student joined.
HAVING ... = ...	Keeps only students whose unique society count matches the total.

Analogy: Imagine there are 6 different colored balls. If someone has exactly 6 balls and all are different colors, they have **all** colors.

Q25. Count the number of societies with more than 5 students enrolled in it.

English Breakdown:

English Fragment

SQL Mapping

“Count the number of societies”

SELECT COUNT(*)

“with more than 5 students enrolled in it”

Derived table / subquery that filters groups
having count > 5

Logical Thinking:

1. **Two-layer problem:**
 - **Layer 1:** Find which societies have > 5 students (this is Q8).
 - **Layer 2:** Count how many such societies exist.
2. You cannot put HAVING directly inside COUNT(*). You must first create the filtered list of societies, then count the rows in that list.
3. **Derived table approach:** Wrap the grouping/filtering query in parentheses, give it an alias, and count its rows.

The Query:

```
SELECT COUNT(*) AS SocietyCount
FROM (
    SELECT SID
    FROM ENROLLMENT
    GROUP BY SID
    HAVING COUNT(RollNo) > 5
) AS PopularSocieties;
```

Execution Flow:

1. **Inner query:** Groups ENROLLMENT by SID, counts students per society, and returns only those SIDs where the count exceeds 5.
 2. **Outer query:** Treats that result as a temporary table named PopularSocieties and counts how many rows it has.
 3. **Final output:** A single number representing how many societies are “popular” (>5 students).
-

Q26. Add column Mobile number in student table with default value '9999999999'.

English Breakdown:

English Fragment	SQL Mapping
"Add column"	ALTER TABLE ... ADD COLUMN
"Mobile number"	MobileNumber VARCHAR(10)
"with default value '9999999999'"	DEFAULT '9999999999'

Logical Thinking:

1. **Schema modification** → Like Q21, this is **DDL**, not DML. You are changing the table's structure, not querying or updating row data.
2. **Default value purpose** → All existing 10 students in the table currently have no mobile number. MySQL will automatically fill this new column with '9999999999' for every existing row.
3. **Data type**: VARCHAR(10) is chosen because mobile numbers can start with zero and aren't used for arithmetic.

The Query:

```
ALTER TABLE STUDENT ADD COLUMN MobileNumber VARCHAR(10) DEFAULT '9999999999';
```

What happens:

1. MySQL locks the STUDENT table briefly.
2. Adds a new column MobileNumber at the end of the row structure.
3. Populates all existing rows with '9999999999'.
4. Future INSERTs that omit MobileNumber will also receive '9999999999'.

Q27. Find the total number of students whose age is > 20 years.

English Breakdown:

English Fragment	SQL Mapping
"Find the total number of students"	SELECT COUNT(*)
"whose age is > 20 years"	WHERE TIMESTAMPDIFF(YEAR, DOB, CURDATE()) > 20

Logical Thinking:

1. **Age calculation trap**: Do **not** use YEAR(CURDATE()) - YEAR(DOB). This only subtracts the year numbers and gives wrong results for birthdays that haven't occurred yet this year.

- Example: If today is May 2026 and DOB is Dec 2005, $2026 - 2005 = 21$, but the person is still 20.

2. **Correct function:** `TIMESTAMPDIFF(YEAR, DOB, CURDATE())` properly computes **complete years** between two dates.
3. **“Total number”** → `COUNT(*)` in the `SELECT`, not just a list of names.

The Query:

```
SELECT COUNT(*) AS StudentsAbove20
FROM STUDENT
WHERE TIMESTAMPDIFF(YEAR, DOB, CURDATE()) > 20;
```

Execution Flow:

1. `TIMESTAMPDIFF(YEAR, DOB, CURDATE())` calculates each student’s exact age in completed years.
 2. `WHERE` keeps only rows where that age exceeds 20.
 3. `COUNT(*)` returns the total count of such students.
-

Q28. Find names of students who were born in 2001 and are enrolled in at least one society.

English Breakdown:

English Fragment	SQL Mapping
“Find names of students”	<code>SELECT DISTINCT s.StudentName</code>
“who were born in 2001”	<code>WHERE YEAR(s.DOB) = 2001</code>
“and are enrolled in at least one society”	<code>JOIN ENROLLMENT (presence in ENROLLMENT proves enrollment)</code>

Logical Thinking:

1. **Two independent conditions joined by AND:**
 - Condition A: Temporal filter → birth year is 2001.
 - Condition B: Relationship filter → must appear in `ENROLLMENT`.
2. **Why JOIN proves “at least one”?** → An `INNER JOIN` only returns students who have matching rows in `ENROLLMENT`. If a student has 3 enrollments, they appear 3 times — hence `DISTINCT` is needed to show their name once.

3. **Alternative:** WHERE YEAR(s.DOB) = 2001 AND EXISTS (SELECT 1 FROM ENROLLMENT e WHERE e.RollNo = s.RollNo) also works and can be more efficient.

The Query:

```
SELECT DISTINCT s.StudentName
FROM STUDENT s
JOIN ENROLLMENT e ON s.RollNo = e.RollNo
WHERE YEAR(s.DOB) = 2001;
```

Execution Flow:

1. JOIN pairs students with their enrollment records. Students with zero enrollments disappear.
 2. WHERE YEAR(s.DOB) = 2001 filters out students born in any other year.
 3. DISTINCT ensures a student enrolled in multiple societies still appears only once.
 4. Final result: Unique names of 2001-born students who are in at least one society.
-

Summary Cheat Sheet (Q22–Q28)

Query	Key Clauses Used	Key Concept
Q22	UPDATE, SET, WHERE, CURDATE()	Conditional updates per row identifier
Q23	CREATE VIEW ... AS SELECT	Virtual table for reusable reporting
Q24	GROUP BY, HAVING COUNT(DISTINCT) = (SELECT COUNT(*))	Relational division — “all” quantifier
Q25	SELECT COUNT(*) FROM (derived table)	Counting filtered groups
Q26	ALTER TABLE ... ADD COLUMN ... DEFAULT	Schema alteration with backfill
Q27	COUNT(*), WHERE TIMESTAMPDIFF(YEAR, ...)	Accurate age calculation
Q28	JOIN, WHERE YEAR(...) =, DISTINCT	Combining date filter with relationship filter

Ready for the final block Q29–Q30 + Admin Commands! Just send the prompt and I'll finish the set.

Kimi:

Here are the **remaining queries (Q29–Q30)**, the **Additional Practice Queries**, and the **Database Administration Commands** — all explained with the extended **English → SQL Clause Map**.

Extended English to SQL Clause Map (Final Concepts)

English Signal	SQL Clause/Keyword	When to Use
“starts with ‘S’ and ends with ‘t’”	WHERE col LIKE 'S%t'	Combine both patterns into one LIKE expression.
“exactly 3” / “precisely N”	HAVING COUNT(...) = N	Exact aggregate match.
“no students enrolled” / “none”	LEFT JOIN ... WHERE ... IS NULL	Same anti-join pattern as Q12.
“average age” / “mean”	AVG(expression)	Aggregate function for arithmetic mean.
“oldest first”	ORDER BY DOB ASC	Earlier date of birth = older person.
“50% or more of capacity”	HAVING (COUNT/Total * 100) >= 50	Percentage formula inside HAVING.
“comma-separated list”	GROUP_CONCAT(col ORDER BY ... SEPARATOR ', ')	String aggregation in MySQL.
“in X but not in Y”	JOIN (for X) + NOT IN subquery (to exclude Y)	Set difference / exclusion pattern.
“month with highest number”	MONTH(date_col) + GROUP BY + ORDER BY DESC LIMIT 1	Extract date part, then find maximum.
“Create user”	CREATE USER 'user'@'host' IDENTIFIED BY 'pwd'	DCL — access control.
“Create role”	CREATE ROLE 'rolename'	DCL — role-based security.
“Grant privileges”	GRANT privs ON db.table TO user/role	DCL — assign permissions.
“Revoke privileges”	REVOKE privs ON db.table FROM user/role	DCL — remove permissions.
“Create index”	CREATE INDEX name ON	DDL — speed up searches.

Q29. Count all societies whose name starts with 'S' and ends with 't' and at least 5 students are enrolled in the society.

English Breakdown:

English Fragment

SQL Mapping

“Count all societies”

SELECT COUNT(*) (needs subquery for scalar count)

“whose name starts with 'S' and ends with 't'”

WHERE s.SocName LIKE 'S%t'

“at least 5 students are enrolled”

HAVING COUNT(e.RollNo) >= 5

Logical Thinking:

1. **“Starts with 'S' and ends with 't'”** → Combine into one pattern: 'S%t'.
 - 'S' = first character.
 - % = anything in the middle.
 - 't' = last character.
2. **Two filters of different types:**
 - Pattern filter (LIKE) → applies to **rows** → goes in WHERE.
 - Enrollment count filter (>= 5) → applies to **groups** → goes in HAVING.
3. **“Count all societies”** → The query in the file groups by society and filters groups. To get a **single number** (how many such societies exist), you technically need to wrap this in a subquery and count the resulting rows (like Q25). The file's query returns the enrollment count per qualifying society.

The Query (as provided):

```
SELECT COUNT(*) AS SocietyCount
FROM SOCIETY s
JOIN ENROLLMENT e ON s.SocID = e.SID
WHERE s.SocName LIKE 'S%t'
GROUP BY s.SocID
HAVING COUNT(e.RollNo) >= 5;
```

For a True Scalar Count (Single Number):

```
SELECT COUNT(*) AS SocietyCount
FROM (
    SELECT s.SocID
    FROM SOCIETY s
    JOIN ENROLLMENT e ON s.SocID = e.SID
    WHERE s.SocName LIKE 'S%'
    GROUP BY s.SocID
    HAVING COUNT(e.RollNo) >= 5
) AS QualifiedSocieties;
```

Execution Flow:

1. WHERE s.SocName LIKE 'S%' filters societies by name pattern before any grouping.
 2. GROUP BY s.SocID bundles enrollments per surviving society.
 3. HAVING COUNT(e.RollNo) >= 5 discards societies with fewer than 5 enrollments.
 4. SELECT COUNT(*) (in the subquery version) counts how many societies made it through.
-

Q30. Display the following information: Society name, Mentor name, Total Capacity, Total Enrolled, Unfilled Seats.

English Breakdown:

English Fragment	SQL Mapping
"Display the following information"	SELECT col1, col2, col3, col4, col5
"Society name, Mentor name"	s.SocName, s.MentorName
"Total Capacity"	s.TotalSeats
"Total Enrolled"	COUNT(e.RollNo)
"Unfilled Seats"	s.TotalSeats - COUNT(e.RollNo)

Logical Thinking:

1. **Multiple computed columns in one row** → List them all in SELECT.
2. **Capacity minus enrolled = unfilled** → Arithmetic in the SELECT clause.
3. **Societies with zero students must still appear** → LEFT JOIN is essential. Without it, empty societies vanish and their "Unfilled Seats" would never be reported.

4. **GROUP BY requirement** → Since SELECT mixes raw columns (SocName, MentorName, TotalSeats) with an aggregate (COUNT), SQL requires you to group by all non-aggregated columns.

The Query:

```
SELECT
  s.SocName,
  s.MentorName,
  s.TotalSeats AS TotalCapacity,
  COUNT(e.RollNo) AS TotalEnrolled,
  s.TotalSeats - COUNT(e.RollNo) AS UnfilledSeats
FROM SOCIETY s
LEFT JOIN ENROLLMENT e ON s.SocID = e.SID
GROUP BY s.SocID, s.SocName, s.MentorName, s.TotalSeats;
```

Execution Flow:

1. LEFT JOIN ensures every society appears, even with no enrollments.
2. COUNT(e.RollNo) counts enrollments per society (returns 0 for empty societies because COUNT ignores NULL).
3. s.TotalSeats - COUNT(...) computes remaining seats. For empty societies: TotalSeats - 0 = TotalSeats.
4. GROUP BY bundles rows correctly so aggregates apply per society.

Additional Practice Queries

“Display students enrolled in exactly 3 societies.”

English Signal	SQL Mapping
“exactly 3”	HAVING COUNT(e.SID) = 3

```
SELECT s.StudentName, COUNT(e.SID) AS SocietyCount
FROM STUDENT s
JOIN ENROLLMENT e ON s.RollNo = e.RollNo
GROUP BY s.RollNo, s.StudentName
HAVING SocietyCount = 3;
```

Logic: Group enrollments by student, then keep only groups where the count equals exactly 3.

“Display societies with no students enrolled.”

English Signal	SQL Mapping
“no students enrolled”	LEFT JOIN ... WHERE e.RollNo IS NULL

```
SELECT s.SocName
FROM SOCIETY s
LEFT JOIN ENROLLMENT e ON s.SocID = e.SID
WHERE e.RollNo IS NULL;
```

Logic: Same anti-join pattern as Q12. LEFT JOIN preserves all societies; WHERE IS NULL keeps only those without a match.

“Find the average age of students in each course.”

English Signal	SQL Mapping
“average age”	AVG(TIMESTAMPDIFF(YEAR, DOB, CURDATE()))
“in each course”	GROUP BY Course

```
SELECT Course, AVG(TIMESTAMPDIFF(YEAR, DOB, CURDATE())) AS AverageAge
FROM STUDENT
GROUP BY Course;
```

Logic: AVG() computes the mean of the age expression across all rows in each course group.

“List students in descending order of their age (oldest first).”

English Signal	SQL Mapping
“oldest first”	ORDER BY DOB ASC

```
SELECT StudentName, DOB, TIMESTAMPDIFF(YEAR, DOB, CURDATE()) AS Age
FROM STUDENT
ORDER BY DOB ASC;
```

Logic: Older people have **earlier** dates of birth. ASC on DOB sorts from 1998 → 2002, putting the oldest at the top.

“Find societies where enrollment is at 50% or more of capacity.”

English Signal	SQL Mapping
“50% or more of capacity”	HAVING (COUNT(e.RollNo) / s.TotalSeats * 100) >=

```

SELECT s.SocName, s.TotalSeats, COUNT(e.RollNo) AS Enrolled,
  ( COUNT(e.RollNo) / s.TotalSeats * 100) AS EnrollmentPercentage
FROM SOCIETY s
JOIN ENROLLMENT e ON s.SocID = e.SID
GROUP BY s.SocID, s.SocName, s.TotalSeats
HAVING EnrollmentPercentage >= 50;

```

Logic: Calculate a percentage inside HAVING. Only groups where the computed percentage is >= 50 survive.

“Display student names with their enrolled society names (comma-separated).”

English Signal

SQL Mapping

“comma-separated”

GROUP_CONCAT(... ORDER BY ...)

```

SELECT s.StudentName, GROUP_CONCAT(so.SocName ORDER BY so.SocName) AS EnrolledSocieties
FROM STUDENT s
JOIN ENROLLMENT e ON s.RollNo = e.RollNo
JOIN SOCIETY so ON e.SID = so.SocID
GROUP BY s.RollNo, s.StudentName;

```

Logic: GROUP_CONCAT() is a MySQL aggregate that joins multiple text values into one string. It only works when you GROUP BY the student.

“Find mentors who guide more than one society.”

English Signal

SQL Mapping

“more than one”

HAVING COUNT(*) > 1

```

SELECT MentorName, COUNT(*) AS SocietyCount
FROM SOCIETY
GROUP BY MentorName
HAVING SocietyCount > 1;

```

Logic: Group by mentor, count how many societies each mentors, filter for counts greater than 1.

“Display students enrolled in ‘NSS’ but not in ‘Dancing’.”

English Signal

SQL Mapping

“in ‘NSS’”

JOIN with WHERE SocName = 'NSS'

English Signal

SQL Mapping

“but not in ‘Dancing’”

AND RollNo NOT IN (subquery for
Dancing)

```
SELECT s.StudentName
FROM STUDENT s
JOIN ENROLLMENT e ON s.RollNo = e.RollNo
JOIN SOCIETY so ON e.SID = so.SocID
WHERE so.SocName = 'NSS'
AND s.RollNo NOT IN (
  SELECT e2.RollNo FROM ENROLLMENT e2
  JOIN SOCIETY so2 ON e2.SID = so2.SocID
  WHERE so2.SocName = 'Dancing'
);
```

Logic: The subquery builds a list of all RollNos in Dancing. The outer query finds NSS students whose RollNo is **not in** that list. This is **set difference**.

“Find the month with highest number of enrollments.”

English Signal

SQL Mapping

“month with highest number”

MONTH(DateOfEnrollment) + ORDER BY DESC
LIMIT 1

```
SELECT MONTH(DateOfEnrollment) AS EnrollmentMonth, COUNT(*) AS EnrollmentCount
FROM ENROLLMENT
GROUP BY MONTH(DateOfEnrollment)
ORDER BY EnrollmentCount DESC
LIMIT 1;
```

Logic: Extract the month number from dates, group by that number, count rows per month, sort descending, and take the top month.

“Display course-wise student count and how many are enrolled in societies.”

English Signal

SQL Mapping

“course-wise”

GROUP BY Course

“how many are enrolled”

COUNT(DISTINCT e.RollNo) on a LEFT JOIN

```
SELECT
  s.Course,
  COUNT(DISTINCT s.RollNo) AS TotalStudents,
  COUNT(DISTINCT e.RollNo) AS EnrolledStudents
```

```
FROM STUDENT s
LEFT JOIN ENROLLMENT e ON s.RollNo = e.RollNo
GROUP BY s.Course;
```

Logic: LEFT JOIN preserves all students. COUNT(DISTINCT s.RollNo) counts unique students per course. COUNT(DISTINCT e.RollNo) counts only those who have enrollment records (non-NULL).

Section II: Database Administration Commands

These are **DCL (Data Control Language)** and **DDL** commands. They don't query data — they manage **who can access** data and **how fast** queries run.

Create User

```
CREATE USER 'student_user'@'localhost' IDENTIFIED BY 'password123';
```

English: “Create a user named student_user who can only connect from this machine (localhost) and must use the password password123 to log in.”

Create Role

```
CREATE ROLE 'society_manager';
```

English: “Create a named role called society_manager. A role is a **bundle of permissions** that can be assigned to multiple users.”

Grant Privileges to a Role

```
GRANT SELECT, INSERT, UPDATE ON student_society_db.ENROLLMENT TO 'society_manager';
GRANT SELECT ON student_society_db.STUDENT TO 'society_manager';
GRANT SELECT ON student_society_db.SOCIETY TO 'society_manager';
```

English: “Give the society_manager role permission to read, add, and modify rows in the ENROLLMENT table, but only read (not modify) the STUDENT and SOCIETY tables.”

Assign Role to User

```
GRANT 'society_manager' TO 'student_user'@'localhost';
```

English: “Give the student_user all the permissions bundled inside the society_manager role.”

Revoke Privileges from a Role

```
REVOKE INSERT ON student_society_db.ENROLLMENT FROM 'society_manager';
```

English: “Remove the INSERT permission from the society_manager role. Anyone with this role can no longer add new rows to ENROLLMENT.”

Create Index

CREATE INDEX idx_student_name **ON** STUDENT(StudentName);

CREATE INDEX idx_enrollment_date **ON** ENROLLMENT(DateOfEnrollment);

English: “Build a lookup structure (index) on the StudentName and DateOfEnrollment columns so that searches using WHERE StudentName = ... or WHERE DateOfEnrollment = ... run much faster.”

Analogy: An index is like the index at the back of a textbook. It takes up extra space and slows down writing slightly, but makes finding specific information dramatically faster.

Show All Indexes

SHOW **INDEX FROM** STUDENT;

SHOW **INDEX FROM** ENROLLMENT;

English: “Display all indexes that exist on these tables, including their names, columns, and types.”

Final Master Cheat Sheet

Query / Command	Key Concept
Q29	LIKE 'S%' + HAVING >= 5 + scalar count via subquery
Q30	Multi-column SELECT with arithmetic + LEFT JOIN
Exactly 3 societies	HAVING COUNT = 3
No students	Anti-join (LEFT JOIN ... IS NULL)
Average age	AVG() aggregate
Oldest first	ORDER BY DOB ASC
50% capacity	Percentage formula in HAVING
Comma-separated	GROUP_CONCAT()
Mentors > 1	HAVING COUNT(*) > 1
NSS not Dancing	NOT IN subquery (set difference)

Query / Command	Key Concept
Highest month	MONTH() + ORDER BY DESC LIMIT 1
Course-wise counts	LEFT JOIN + COUNT(DISTINCT)
CREATE USER	DCL — identity creation
CREATE ROLE	DCL — permission bundling
GRANT / REVOKE	DCL — permission assignment/removal
CREATE INDEX	DDL — query performance optimization

That covers **every query and command** from your .sql file. If you want me to create a **printable one-page summary** or **flashcard-style revision notes** from all 30+ queries, just ask!
